Claude

THINK
BUILD
COORDINATE
EVOLVE

Claude 103

Agents, Sub-Agents, MCP, and Tips & Tricks

ORCHESTRATE / DELEGATE / CONNECT / OPTIMIZE



Agents

HANDLE
COMPLEX TASKS



Sub-Agents

DIVIDE
AND SPECIALIZE



MCP

CONNECT
TOOLS & DATA



Tips & Tricks

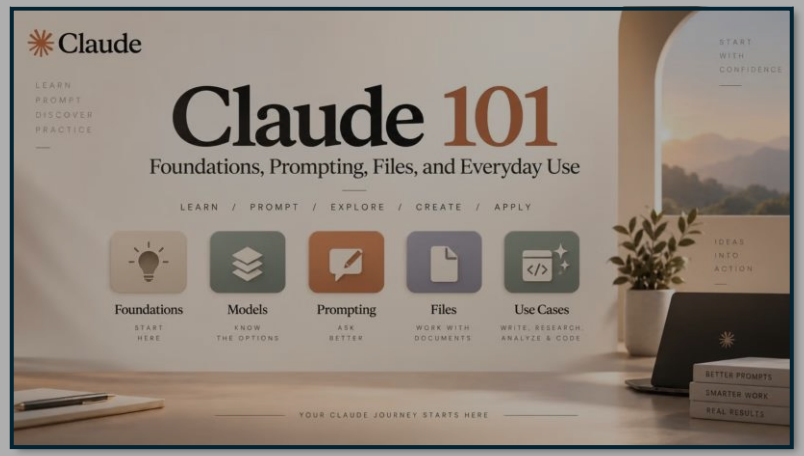
WORK
SMARTER

TASKS
INTO
WORKFLOWS

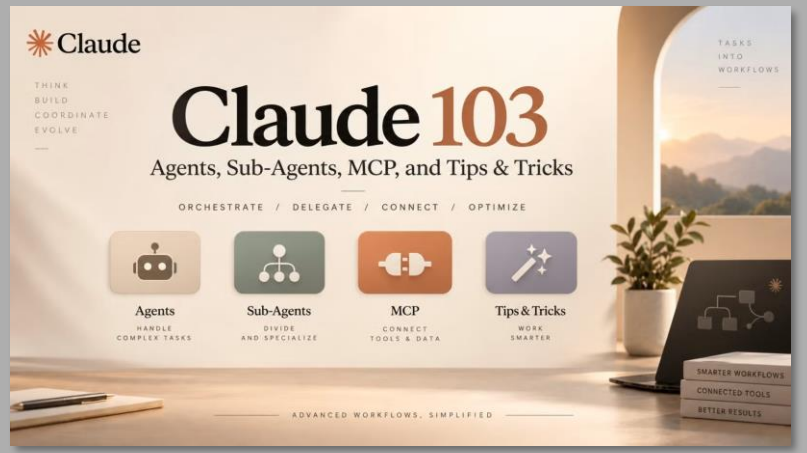


ADVANCED WORKFLOWS, SIMPLIFIED

Three sessions, one journey







101 • Foundations

What Claude is and where it fits
Model families
Prompting and context
Documents and files

102 • Workspace

Settings, profile, and preferences
Projects: files, instructions, context
Skills: reusable workflows
Artifacts: docs, code, diagrams

103 • Advanced

Agents + sub-agents
Coordinating and delegating work
Model Context Protocol (MCP)
Automation + integrations

Agenda

- Claude 101 + 102 recap
- Agents
- Sub-agents
- Coordinating work
- Model Context Protocol (MCP)
- Tips & tricks
- Three live sub-agent demos
- Q&A



Where we left off

1 · Prompt

Role, task, context, constraints, output

2 · Project

Files and instructions for one workstream

3 · Skill

A procedure Claude loads only when needed

4 · Artifact

A deliverable you can iterate on and share



Context window

Still the limit. Tonight agents fill it fast, and sub-agents protect it.



Who drives

In 101 and 102 you drove every step. Tonight Claude takes the wheel for a while.



Tonight

Agents act, sub-agents divide the work, and MCP connects your tools.

Agents

Claude works toward a goal: it plans, uses tools, checks the results, and repeats until the job is done.

From answering to doing



In a chat

- You ask; Claude answers once
- You copy, paste, and run things yourself
- You carry the context between steps
- Great for drafts, questions, and analysis
- One message in, one response out



With an agent

- You set a goal; Claude works toward it
- It calls tools: search, files, terminal, browser
- It checks results and picks the next step
- Great for multi-step work you'd otherwise babysit
- You review the outcome and approve risky steps

The agent loop

Agent Loop

1 · Gather

Read files, search, and ask for what it needs

2 · Plan

Break the goal into the next few steps

3 · Act

Call a tool: edit a file, run a command, fetch a page

4 · Verify

Check the result, then loop again or stop

Tools

What it can do: search, read, edit, run, call APIs. No tool, no action.

Context window

Every tool result lands in the context. Long jobs fill it quickly.

Stop conditions

Say what done looks like: tests pass, report written, or a question for you.

Where Claude works as an agent

AGENTS

Same loop everywhere. What changes is the tools it has and where it runs.

- Pick the surface by where the work lives: the web, your repo, or your own app.
- Check your plan: some agent features are paid-only.
- Claude Code and the Agent SDK are where developers build.
- Connectors bring your own tools into any of them.

Research

Searches many sources, then reports with citations

Claude Code

Edits code and runs commands in your repo

Claude in Chrome

Navigates and fills in web pages for you

Agent SDK

Builds your own agent on the same harness

Connectors (MCP)

Your tools and data, covered in Part 3

Are agents available on the free tier?

PLANS

Short answer: no. Sub-agents live in Claude Code, and the Free plan doesn't include Claude Code.

Free plan: no

Chat on claude.ai only. No Claude Code, so no sub-agents and no custom agents.

Pro, Max, and Ultra: yes

Claude Code is included: terminal, IDE, desktop app, and web. Built-in and custom sub-agents work.

Team and Enterprise: yes

Claude Code is included on these plans. Admins decide who gets access.

API (Console): pay as you go

No subscription: sign in with a Claude Console account and pay per token. Not free, but no monthly plan.

Brief an agent like a new teammate

AGENTS

The five-part prompt from 101 becomes a delegation brief.

1	**GOAL**	“Find why member orders total wrong after Friday's release…”
2	**CONTEXT**	“C# web app; start with PriceCalculator.cs and its tests…”
3	**BOUNDARIES**	“Read anything; ask before editing; never touch production…”
4	**DONE WHEN**	“Bug found, fix proposed, dotnet test passes…”
5	**REPORT**	“Summarize the evidence, the change, and what you didn't check.”

Agents fill gaps with guesses. Every line you write here is one less guess.

Choose how much autonomy to give

AGENTS

Claude Code asks before risky actions. Shift+Tab switches modes as your trust grows.

Ask first (default)

Claude asks before each file edit or command. Best while you're learning the tool.

Plan mode

Read-only research and a written plan. Nothing changes until you approve it.

Accept edits

File edits go through; shell commands still ask. Good for a well-scoped task you trust.

Allow lists

Pre-approve safe commands, such as dotnet test, in settings so routine steps don't interrupt you.

Sub-agents

Specialists the main agent delegates to, each with its own context, instructions, and tools.

What a sub-agent is

A sub-agent is a specialist Claude the main agent hands a task to. It works in its own context and returns only a summary.

The main agent decides when to delegate, based on each sub-agent's description.

Claude Code ships built-in ones, such as Explore and Plan, and you can add your own.

Run /agents to see the built-in and custom agents you have.



Own context

Starts clean, no chat history



Own prompt

A role and its steps



Own tools

Only what it needs



Own model

Haiku, Sonnet, or Opus

Why split the work

SUB-AGENTS

Same reason teams have specialists: focus beats one overloaded generalist.

Clean context

Noisy searches and logs stay in the sub-agent; only the summary comes back

Specialists

A focused prompt for one job: proofreader, reviewer, test writer

Parallel

Independent tasks run at the same time and finish sooner

Least privilege

A reviewer can read but not edit; a cheaper model can do the grunt work

Rule of thumb: if a step would flood the main context, delegate it.

Built-in vs custom agents



Built-in: ready to use

- Explore: read-only search across your code
- Plan: read-only research while in plan mode
- general-purpose: every tool, for multi-step work
- Also: claude-code-guide, statusline-setup
- Nothing to install; Claude picks them on its own



Custom: you write them

- A Markdown file, or let /agents draft one for you
- Your prompt, your tool list, your model
- Per project or personal across all projects
- Called by its description, or by name
- Tonight's demos: proofreader and code-reviewer

Anatomy of a sub-agent

A Markdown file in .claude/agents/, or create one with the /agents command.

```
---
name: proofreader
description: Checks text files for spelling,
  grammar, and wording mistakes. Reports only.
tools: Read
model: haiku
---
You are a careful proofreader.
1. Read the file you are given.
2. List each mistake: line, wrong text, fix.
3. Do not rewrite or edit any file.
4. End with: "Found N problems."

File: .claude/agents/proofreader.md
```

- Looks like a 102 skill: front matter, then instructions.
- The description tells the main agent when to delegate.
- tools limits what it can touch; leave it out to inherit all.
- model picks a cheaper or stronger model per job.

Skills, tools, and permissions

SUB-AGENTS

Front-matter fields decide what a sub-agent knows, what it can touch, and what it may do.

Tools

tools: is an allowlist. Leave it out to inherit every tool. disallowedTools: removes some.

Example:
tools: Read, Grep, Glob

Permissions

permissionMode: default, acceptEdits, plan, and more. Your session's mode can override it.

Example:
permissionMode: plan

Skills

Not inherited. List them in skills: to preload the full skill at startup.

Example:
skills: house-style

MCP and more

mcpServers: gives it specific MCP tools. Also model, maxTurns, memory, hooks.

Example:
model: haiku

Least privilege: give each agent only what its job needs.

Project vs global agents

Project agent

- File: .claude/agents/<name>.md in the repo
- Works only inside that project
- Commit it so the whole team gets it
- Wins if a global agent has the same name
- Demos 2 and 3 use project agents

Global (user) agent

- File: ~/.claude/agents/<name>.md
- Windows: %USERPROFILE%\.claude\agents\
- Works in every project on your machine
- Personal: not shared through git
- Pick the scope when you create it in /agents

Don't start from scratch: a sub-agent library

COMMUNITY

The definitive collection of Claude Code subagents, specialized AI assistants designed for specific development tasks.

- github.com/VoltAgent/awesome-claude-code-subagents
- 160+ sub-agents in 10 categories, MIT license
- Install a category as a plugin or copy one .md file into .claude/agents/ or ~/.claude/agents/
- Read each file first: the maintainers don't audit them
- Trim the tools list to what you need

Core development

api-designer, frontend-developer

Language specialists

typescript-pro, python-pro

Infrastructure

docker-expert, kubernetes-specialist

Quality and security

code-reviewer, security-auditor

Meta and orchestration

agent-organizer, context-manager

Four ways to divide the work

PATTERNS

Pick the pattern that matches how the task breaks apart.

Fan-out

Same question, many sources: one sub-agent per service, log, or repo, in parallel

Most common ↑

Pipeline

Each step feeds the next: analyze, then fix, then test

Reviewer

One writes, another critiques with fresh eyes and a different prompt

Orchestrator

The main agent plans, assigns the pieces, and merges the results

Start with one agent. Add sub-agents when one context gets crowded.

Delegate, coordinate, combine

1 · Decompose

Split the goal into independent pieces

2 · Delegate

Give each sub-agent a written brief

3 · Collect

Each returns a short, structured summary

4 · Combine

The main agent merges, reconciles, and decides



Hand-offs in writing

Sub-agents can't see your chat. Everything they need goes in the brief.



Summaries, not transcripts

Ask for a fixed format: findings, evidence, confidence. Easy to merge.



Check the merge

Disagreement is a signal. Ask the main agent to settle it with evidence.

When to use sub-agents, and when not to



Good fit

- Research across many files, logs, or services
- Independent tasks that can run in parallel
- Noisy work where you only need the conclusion
- Second-opinion reviews with a different prompt
- Jobs that need tighter tool permissions



Poor fit

- Small tasks: delegation has overhead
- Tightly coupled edits to the same files
- Steps that need your back-and-forth
- Work where the whole history matters
- Watch cost: every sub-agent uses tokens

Demo



Sub-agents

Three quick demos: store readers, proofreader, code reviewer

DEMO 1 • Three stores, three readers

LIVE DEMO

Scenario: customer comments from three stores. Three sub-agents each read one store at the same time.

Step 1 • Set up

Open a terminal in demo1-parallel-readers and run claude. Three files in feedback/, no agent file needed.

Step 2 • Ask

Ask:

“Use three sub-agents in parallel, one per file in the feedback folder. Each returns the top complaint and top compliment. Then give me one table.”

Step 3 • Show

Three tasks start at once (Ctrl+O expands them). One table comes back comparing Nashua, Boston, and Portland.

Goal: show parallel work and clean context. The main agent never reads the raw comments.

DEMO 2 • The proofreader

LIVE DEMO

Scenario: a typo-spotting specialist that can read but not edit, running on the fast, cheap model.

Step 1 • Inspect

In demo2-proofreader, run /agents. Open proofreader under project agents. Point at tools: Read and model: haiku.

Step 2 • Delegate

Ask:

“Use the proofreader sub-agent to check welcome.md. Don't fix anything yet.”

Step 3 • Fix

It lists 7 mistakes. Then say, “Now fix them.” The main agent edits, because the proofreader can't.

Goal: show the anatomy: a name, a description, one tool, and a cheaper model.

DEMO 3 • Fresh-eyes code review

LIVE DEMO

Scenario: a read-only reviewer checks PriceCalculator.cs; the main agent fixes what it finds.

Step 1 • Delegate

In demo3-code-reviewer, ask:

“Ask the code-reviewer sub-agent to review PriceCalculator.cs. Then fix every bug it finds and show me the diff.”

Step 2 • Report

The reviewer finds the 1% member discount, the exactly-$100 shipping edge case, and a divide-by-zero on an empty cart.

Step 3 • Fix

Approve the edit, or press Shift+Tab for accept-edits first. The main agent shows the diff.

Goal: show the reviewer pattern. Writer and reviewer stay separate, like a pull request.

MCP

Model Context Protocol: one standard way to connect Claude to your tools, APIs, and data.

What MCP is

The Model Context Protocol is an open standard for connecting AI apps to tools and data. Build a server once; any MCP client can use it.

Introduced by Anthropic in 2024; now an open standard used across the industry.

Think USB-C for AI: one plug instead of a custom integration per app.

In claude.ai, MCP servers show up as connectors.



Tools

Actions Claude can call



Resources

Data Claude can read



Prompts

Templates a server offers



Open standard

Any client, any server

How MCP works

1 · Host

The app you use: claude.ai, Claude Desktop, Claude Code

2 · Client

One connection per server, inside the host

3 · Server

Exposes tools, resources, and prompts

4 · Your system

An API, database, file share, or SaaS app



Local (stdio)

The server runs on your machine as a process. Good for files and dev tools.



Remote (HTTP)

The server runs at a URL. Good for SaaS and shared team tools.



Sign-in

Remote servers usually use OAuth, so Claude acts with your permissions.

Where you plug MCP in

MCP

Same protocol, different front doors.

Connectors in claude.ai

Settings → Connectors. Pick from the directory or add a remote server URL as a custom connector.

Claude Desktop

Desktop extensions install local servers in a click, for files and apps on your machine.

Claude Code

claude mcp add on the command line, or a .mcp.json file in the repo shared with your team.

Agent SDK and API

Your own agents and apps connect to the same servers in code.

Add an MCP server to Claude Code

MCP

One command per server. Check what's connected with /mcp inside a session.

```
# Remote server over HTTP (public, no sign-in)
claude mcp add --transport http mslearn \
  https://learn.microsoft.com/api/mcp

# Local server over stdio (C# server, next slide)
claude mcp add ops-tools -- \
  dotnet run --project ./OpsToolsMcp

# Share with the team: writes .mcp.json
claude mcp add --scope project ops-tools \
  -- dotnet run --project ./OpsToolsMcp

# See what is connected
claude mcp list
```

- Everything after -- is the command that starts the server.
- Scopes: local (just you), project (.mcp.json), user (all your projects).
- Commit .mcp.json so the team gets the same tools.
- Keep secrets out of it; use environment variables.

Build your own MCP server in C#

.NET

The official C# SDK turns a static method into a tool Claude can call.

```
// dotnet add package ModelContextProtocol
var builder = Host.CreateApplicationBuilder(args);
builder.Services.AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();
await builder.Build().RunAsync();

[McpServerToolType]
public static class OpsTools
{
  [McpServerTool, Description("Recent deployments")]
  public static string GetDeployments(int hours) =>
      ChangeLog.Since(TimeSpan.FromHours(hours));
}
```

- The Description is what Claude reads to decide when to call the tool.
- Parameters become the tool's input schema automatically.
- Log to stderr: stdout carries the protocol.
- The same server works in Claude Code, Claude Desktop, and other MCP clients.

Treat MCP servers like code you run

Before you connect

- Use official or well-known servers you can inspect
- Read which tools it exposes and what they change
- Grant the narrowest scopes and accounts
- Prefer read-only tools until you trust it
- Follow your company's AI and security policy

While it runs

- Tool results can carry prompt injection
- Approve write actions; don't auto-allow everything
- Keep secrets in env variables, not config files
- Too many tools crowd the context window
- Remove servers you don't use anymore

Tips & Tricks

Shortcuts, prompting techniques, and automation for everyday agent work.

Six Claude Code shortcuts

1 Plan before you build

Shift+Tab → plan mode. Nothing changes until you approve.

2 Interrupt early

Esc stops Claude. Esc Esc rewinds to an earlier point.

3 Point at files

@PriceCalculator.cs pulls it into context.

4 Run a command inline

! dotnet test runs it and shares the output.

5 Keep context lean

/clear between tasks, /compact long ones, /context to check.

6 Write it down once

/init creates CLAUDE.md. Commit it for the team.

Brief the agent, don't just ask

PROMPTING

The 101 lesson again, with higher stakes: an agent acts on its guesses.

Vague

“Fix the checkout bug.”

Problems

- No evidence to start from
- No boundaries
- No definition of done
- No way to verify

Specific

“Since Friday's release, member orders total wrong. Start with PriceCalculator.cs and its tests. Plan first and wait for my OK. Don't change the public API. Done means dotnet test passes and you show the diff.”

Automate the repetition

AUTOMATION

Things you'd type every session belong in files and hooks.

CLAUDE.md

Project memory: build commands, conventions, and rules every session reads

Commands + skills

Save a prompt as /postmortem; your 102 skills work here too

Hooks

Run your script at set moments, such as dotnet format after every edit

Headless mode

claude -p "..." in scripts and CI, such as a pipeline that reviews pull requests

Start here ↑

Prompt it once • Write it down • Script it

Which tool for which job?

TOGETHER

103 adds two new answers to the 102 question.

Prompts handle one task, right now.

Projects hold the context for one stream of work.

Skills hold procedures you repeat across projects.

Sub-agents take one part of the job in their own context.

MCP servers connect Claude to your tools and data.

Prompt

This task, right now

Project

What we're working on

Skill

How we do this task

Sub-agent

Who does this part

MCP server

What Claude can reach

Claude 103 – Key Takeaways

- Brief an agent like a new teammate
- Plan first, then let it act
  - Verify with tests, not vibes
- Use sub-agents for focused, parallel work
- Connect tools with MCP, and trust them like code





Claude

Thank you · Claude 101, 102, and 103 · Slides and demo code on GitHub



References

Daniel Colón LinkedIn
https://www.linkedin.com/in/danielecolon/

Presentation Slides/Code
https://github.com/danielecolon/Claude

Claude Code sub-agents
https://code.claude.com/docs/en/sub-agents

Claude Code and MCP
https://code.claude.com/docs/en/mcp

Model Context Protocol
https://modelcontextprotocol.io

MCP C# SDK
https://github.com/modelcontextprotocol/csharp-sdk

Building effective agents
https://www.anthropic.com/engineering/building-effective-agents

Awesome Claude Code subagents
https://github.com/VoltAgent/awesome-claude-code-subagents

Q&A

Questions, ideas, and what you'd like to see next.